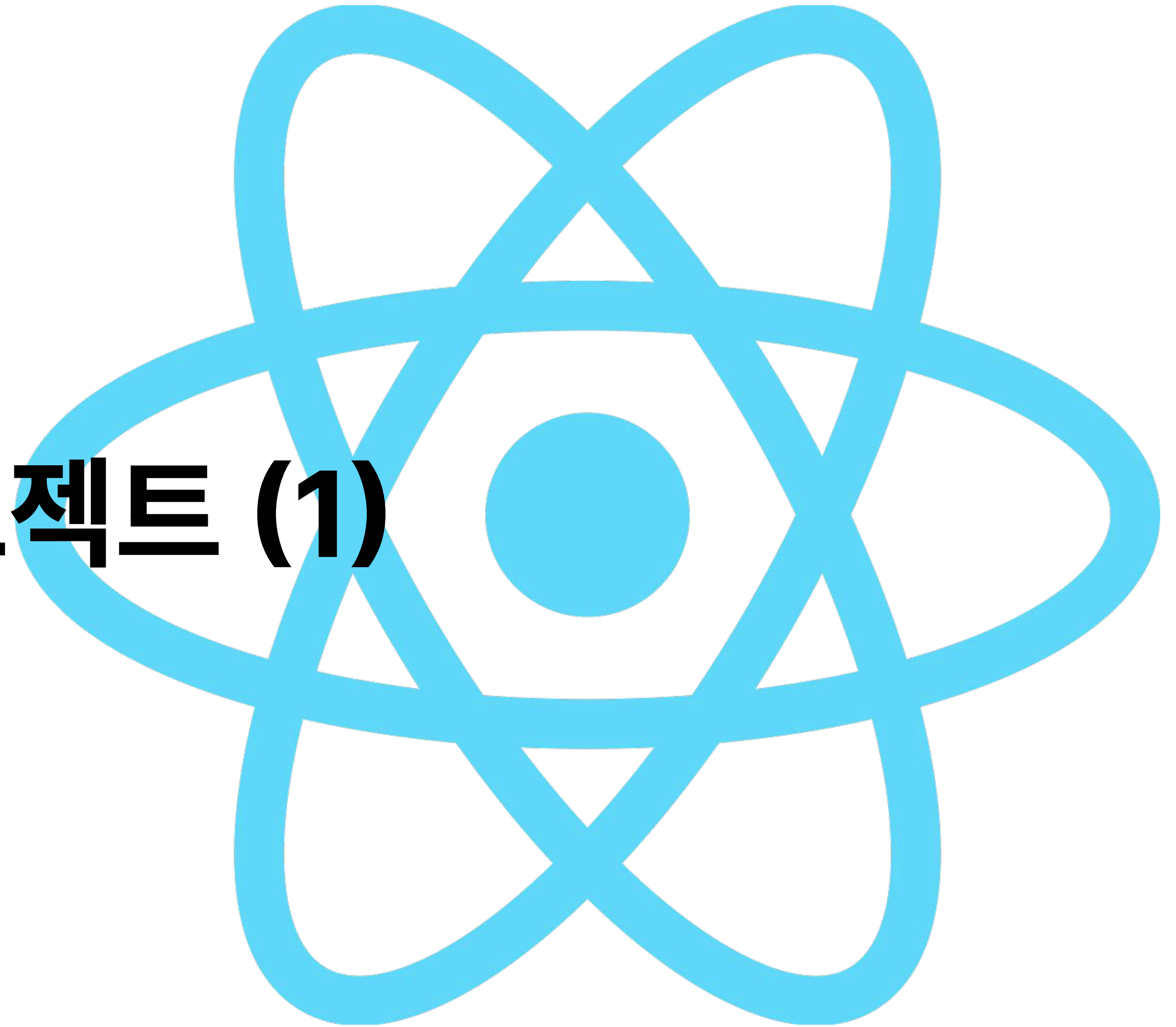


React 쇼핑몰 프로젝트 (1)

EY 기업맞춤형 AI+X 융복합과정 보조강사 정승민



1일차 목차

01 1교시 · React 시작하기

React의 필요성 · 특징 이해와 개발 환경 · 도구 준비

02 2교시 · JSX로 상점 첫 화면 만들기

파일 역할 · JSX 이해와 상점 이름 · 상품 · 가격 표시

03 3교시 · 컴포넌트로 상품 목록 만들기

Props로 상품 데이터 전달과 map으로 상품 카드 목록 표시

04 4교시 · State로 검색과 필터 구현하기

입력값의 State 관리와 검색 · 카테고리 · 빈 결과 화면 연결

React 학습에 필요한 기초 지식



HTML · 화면의 구조

태그, 요소의 중첩 구조,
입력 요소



CSS · 화면의 모양

선택자, 클래스,
기본 배치와 스타일



JavaScript · 화면의 동작

변수 · 함수 · 객체
· 배열 · 이벤트

→ React는 사용자 인터페이스(UI)를 만드는 JavaScript 라이브러리

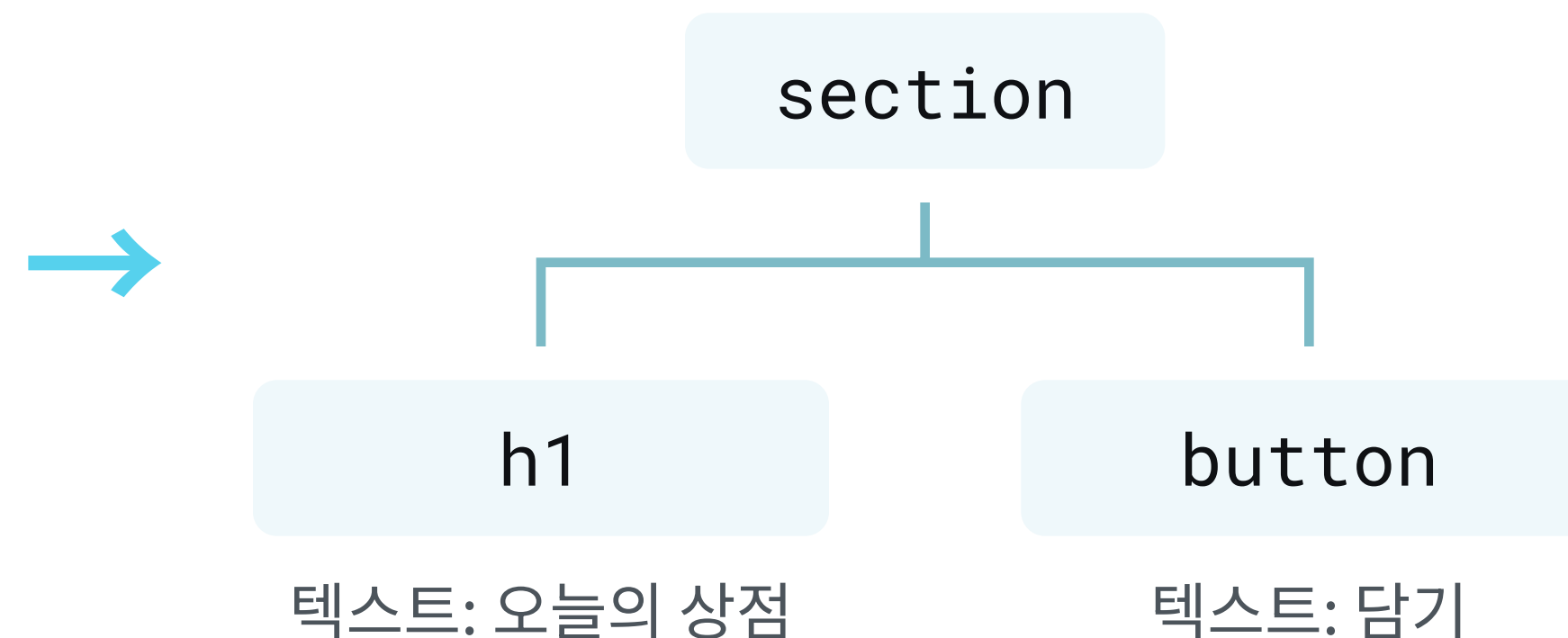
DOM : 브라우저가 다루는 문서 구조

DOM(Document Object Model)은 HTML 문서를 객체와 트리 구조로 표현한 모델임

HTML · 작성한 문서

```
<section>
  <h1>오늘의 상점</h1>
  <button>담기</button>
</section>
```

DOM · 문서의 일부 구조



브라우저 화면

오늘의 상점

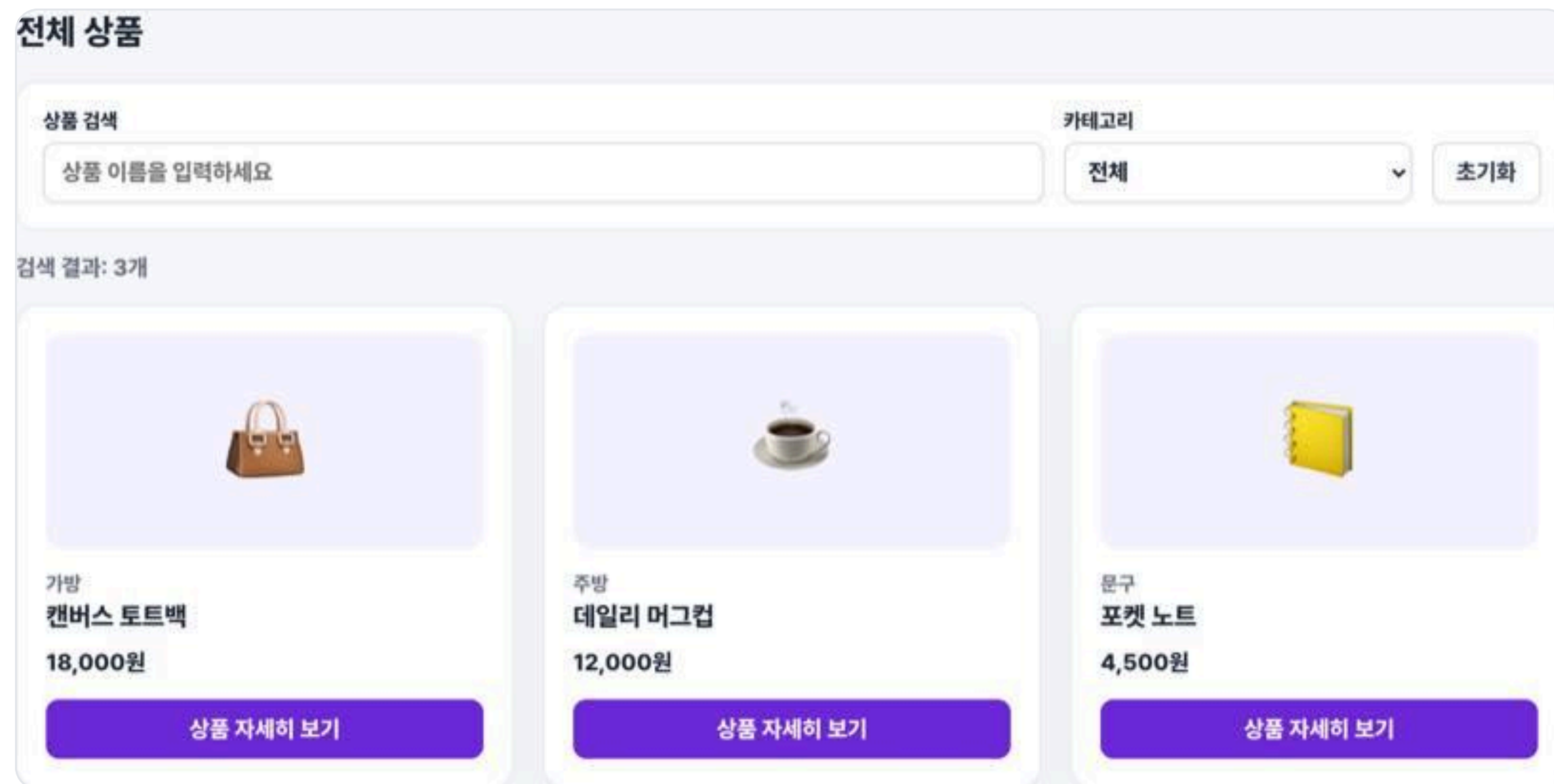
담기

JavaScript는 DOM을 읽기·변경 가능
React가 DOM을 생성·갱신하고 브라우저가 화면 표시

React 도입의 필요성

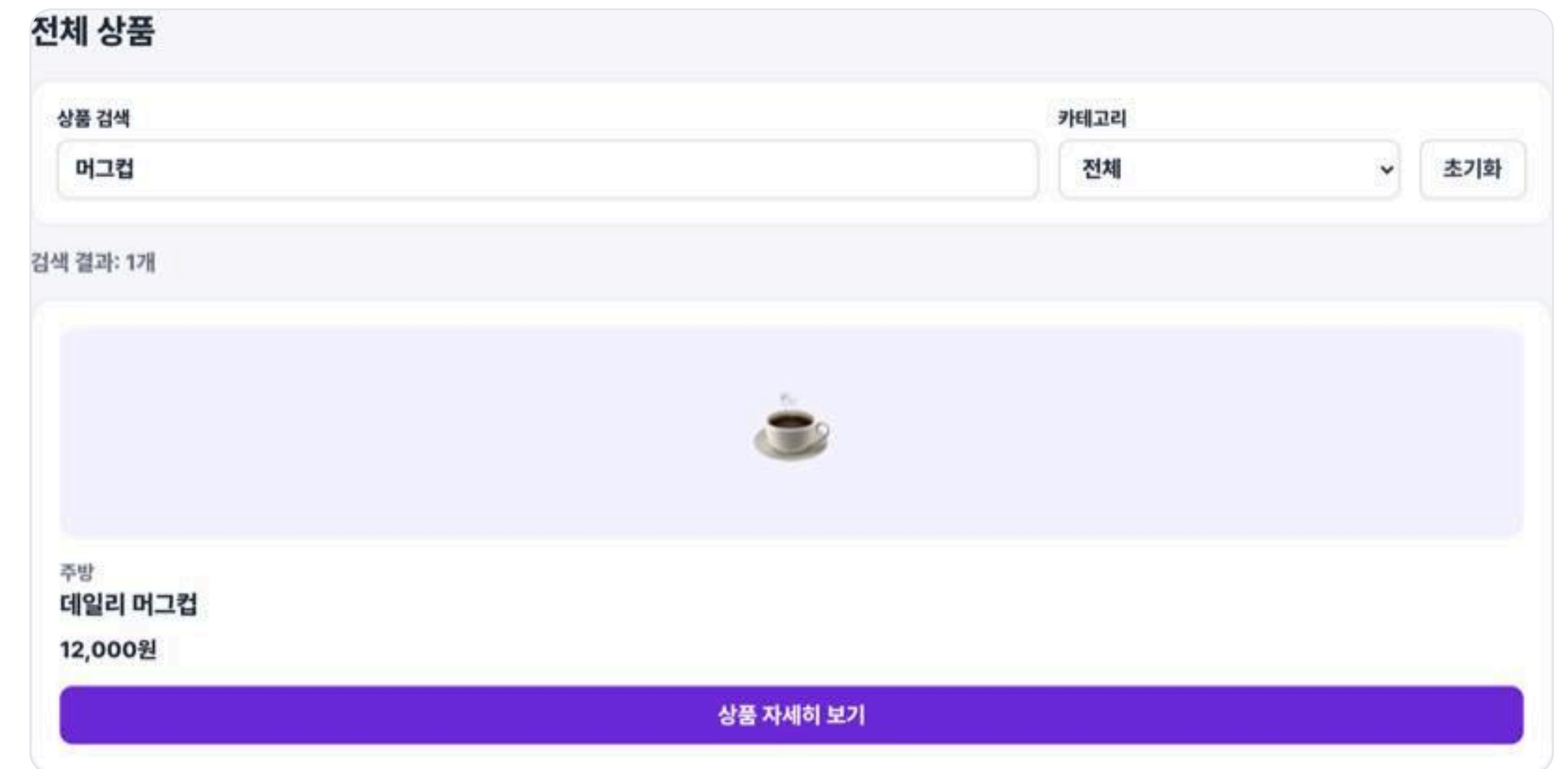
사용자 행동에 따라 달라지는 화면

검색 전 · 전체 상품 3개



검색어 없음 → 전체 상품 표시

검색 후 · ‘머그컵’ 검색



검색 결과 1개 → 머그컵만 표시

React는 State를 기준으로 연결된 화면의 변화를 관리함

특징 1 - 컴포넌트 기반 구조

01 컴포넌트 (Component)

고유한 모양과 로직을 가진 UI 구성 단위
이 교안에서는 JSX를 반환하는
JavaScript 함수로 작성

02 조합성 (Composition)

컴포넌트 중첩·조합으로
하나의 화면 구성

03 재사용성 (Reusability)

같은 컴포넌트를 여러 위치에서
서로 다른 데이터와 함께 재사용



특징 2 - 선언적 UI

명령형은 DOM을 직접 수정하고, 선언형은 State에 따른 화면 규칙 작성

예시: 담기 버튼 클릭 → 수량 증가 · 빈 장바구니 안내 숨김

바닐라 JS · 직접 DOM 갱신

cart.js · 비교 예시

```
// HTML에 #count, #empty, #add 준비
const countEl = document.querySelector("#count");
const emptyEl = document.querySelector("#empty");
let count = 0;
function updateCart() {
  countEl.textContent = `장바구니 ${count}개`;
  emptyEl.hidden = count > 0;
}
document.querySelector("#add").onclick = () => {
  count += 1;
  updateCart();
};
updateCart();
```

이점: 별도 UI 라이브러리 없이 시작

부담: 여러 화면 요소의 갱신을 직접 맞춤

React · 상태로 화면 선언

src/App.jsx · 비교 예시

```
// useState를 import한 컴포넌트 내부
const [count, setCount] = useState(0);
return (
  <>
    <p>장바구니 {count}개</p>
    {count === 0 && (
      <p>장바구니가 비었습니다</p>
    )}
    <button onClick={() => setCount(count + 1)}>
      담기
    </button>
  </>
);
```

이점: 상태에 맞는 화면 규칙을 한곳에 작성

부담: JSX·State와 개발 환경 학습

특징 3 - State: 컴포넌트가 기억하는 데이터

01 State의 의미

컴포넌트가 기억하는, 바뀔 수 있는 데이터

예: 담긴 상품 수 count, 입력한 검색어 searchTerm

02 State 기반 화면 구성

count = 0 → '장바구니 0개'와 빈 장바구니 안내 표시

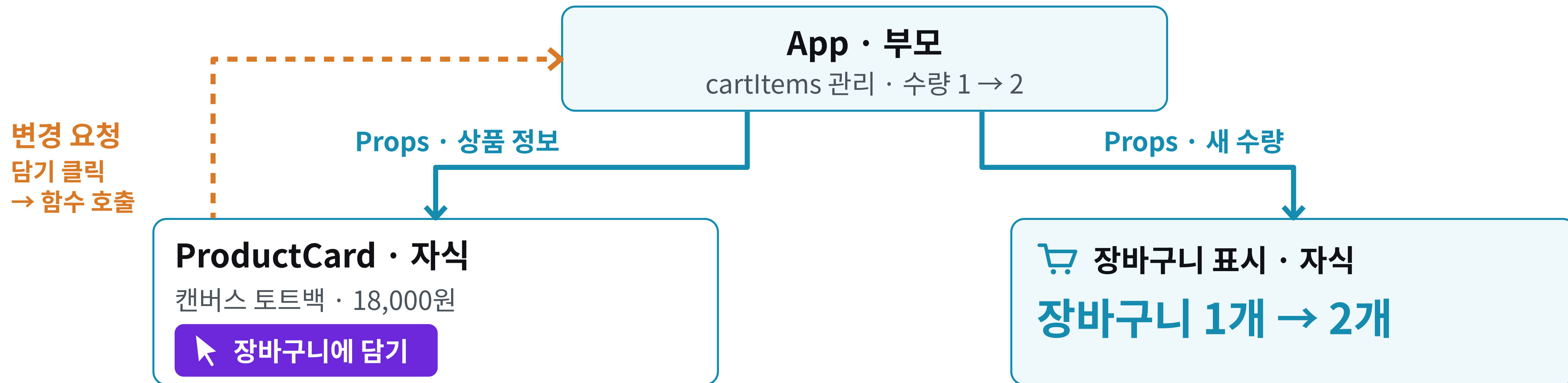
count = 1 → '장바구니 1개', 빈 안내는 표시하지 않음

03 사용자 입력에 따른 State 변경

담기 클릭 → count를 0에서 1로 변경 → 새 값에 맞는 화면 반영

빈 장바구니는 count === 0으로 판단. 별도 State 저장은 불필요

특징 4 - 단방향 데이터 흐름



데이터는 아래로, 변경은 상태를 가진 부모가 처리

단방향 흐름으로 얻는 이점

상태의 관리자가 분명

장바구니 상태는 App에서 관리
ProductCard는 변경 함수 호출

변화를 예측하기 쉬움

상태 변경 → 새 Props → 화면
변화의 순서를 따라가기 쉬움

오류를 찾기 쉬움

함수 호출 → State → Props
값이 어긋난 지점을 확인

React 도입 판단 기준

예시 1 · 수업 안내 페이지

... 수업 안내

React 특강 안내

수업 주제	React로 쇼핑몰 만들기
준비물	노트북
문의	담당 강사

정보 안내가 중심 · 화면 변화가 적음
HTML·CSS로도 충분할 수 있음

예시 2 · 미니 쇼핑몰

... 오늘의 상점

장바구니 (2)

검색 결과: 1개

 **데일리 머그컵**
12,000원

담기

장바구니: 머그컵 2개 · 합계 24,000원

검색 · 필터 · 수량에 따라 화면 변경
컴포넌트 재사용과 State 관리에 도움

화면 변화가 많고 연결된 요소가 늘수록 React 활용에 이점이 있음

도입 시 JSX·State 문법과 개발 환경 학습 비용도 고려

SPA · 하나의 문서에서 화면 전환

SPA(Single Page Application)는 한 웹 문서를 유지하며 JavaScript로 화면 내용을 바꾸는 앱 구조.

상품 화면

오늘의 상점

상품

장바구니 (1)

로그인

상품 목록

데일리 머그컵 · 12,000원

담기



장바구니 화면

오늘의 상점

상품

장바구니 (1)

로그인

장바구니

데일리 머그컵 1개

합계 12,000원

이점 · 문서 전체 재로딩 없이 이동

앱 안에서 화면 전환과 상호작용이 이어짐

고려 · 이동과 상태를 관리

URL·뒤로 가기·State 관리가 필요
필요한 데이터는 서버에 별도로 요청 가능

CSR · 브라우저에서 JavaScript로 UI 생성

CSR(Client Side Rendering)은 브라우저에서 JavaScript를 실행해 DOM과 화면 내용을 만드는 렌더링 방식.



SPA와 CSR은 서로 다른 개념

SPA: 같은 문서에서 화면을 전환하는 구조
CSR: 브라우저에서 UI를 만드는 방식

React의 렌더링 선택지

Vite + React의 기본 앱은 CSR 기반 SPA
React는 SSR(서버 렌더링)도 지원

개발 환경 이해 · Node.js, npm, Vite



Node.js · 실행 환경

브라우저 밖에서 JavaScript를
실행하는 런타임
서버 · 파일 처리 · 개발 도구에 활용



npm · 패키지 관리

다른 개발자가 만든 코드(패키지)를
설치하고 버전 · 의존성을 관리
프로젝트 명령 실행도 지원



Vite · 개발 · 빌드 도구

개발 서버로 빠르게 결과 확인
코드 변경을 화면에 즉시 반영
배포할 파일로 코드를 변환 · 묶기

런타임(Runtime)의 의미

코드를 실행하는 데 필요한 환경임

브라우저: 웹 화면의 JavaScript 실행 · Node.js: 브라우저 밖의 JavaScript 실행

Node.js와 npm 설치

01 공식 사이트에서 설치 파일 받기

nodejs.org/en/download → LTS → Windows → Installer (.msi)
PC에 맞는 아키텍처 선택: 일반적인 Intel·AMD PC는 x64

02 설치 마법사 진행

Next → 사용권 동의 → 기본 설치 위치와 구성 유지 → Install
npm package manager와 Add to PATH를 포함한 상태로 설치

03 설치 후 새 명령 프롬프트 열기

Finish → 기존 터미널 닫기 → 시작 메뉴에서 cmd 실행
열려 있던 VS Code도 다시 실행

04 버전 번호로 설치 확인

오른쪽 두 명령에서 각각 버전 번호가 출력되면 준비 완료
명령을 찾지 못하면 설치 상태와 새 터미널 사용 여부 확인

명령 프롬프트(cmd)

터미널 · 명령 프롬프트(cmd)

```
node -v  
npm -v
```

두 명령 모두 버전 출력

Node.js와 npm이 함께 준비된 상태

설치 이후에 확인

이미 열려 있던 터미널 대신
새 창에서 확인

현재 버전 · 2026.10.06 기준

최신(Current): v26.10.0
설치 안내(LTS): v24.21.0

VS Code와 터미널 준비

01 VS Code 설치

code.visualstudio.com/download → Windows User Installer 다운로드
설치 파일 실행 → 사용권 동의 → 안내에 따라 설치 완료

02 실습 폴더 열기

VS Code → 파일(File) → 폴더 열기(Open Folder)
실습용 폴더 선택. 프로젝트 생성 후에는 mini-shop 폴더 열기

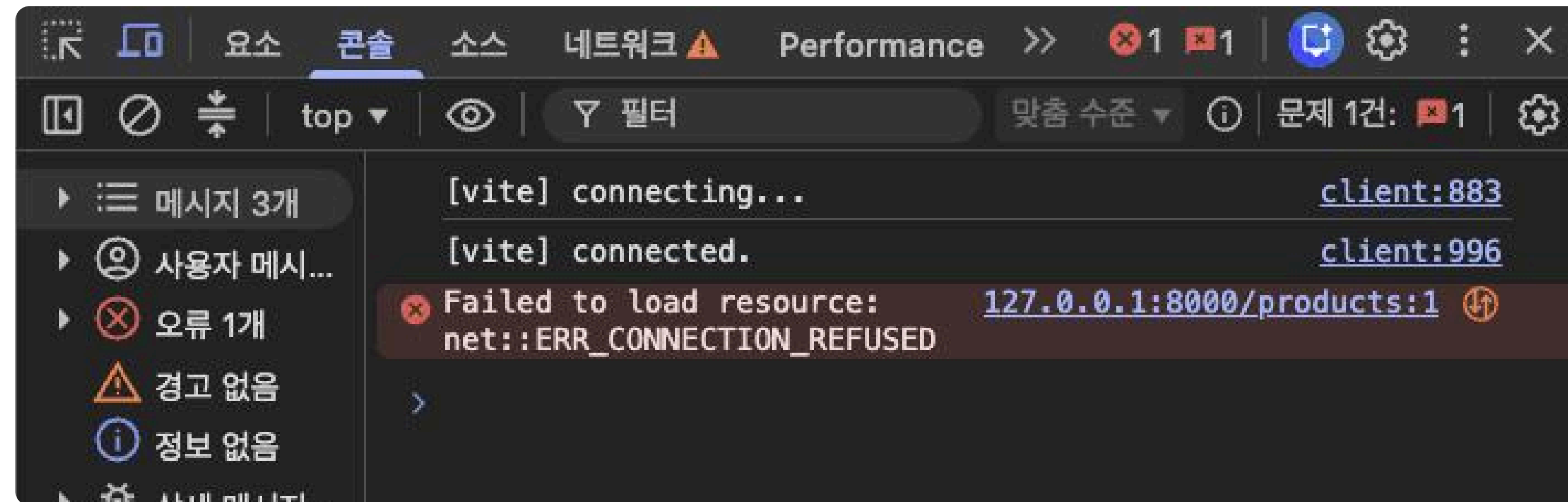
03 명령 프롬프트를 기본 터미널로 선택

Ctrl+Shift+P → Terminal: Select Default Profile → Command Prompt
터미널(Terminal) → 새 터미널(New Terminal)로 다시 열기

04 현재 위치와 사용 목적 확인

프로젝트 명령은 package.json이 있는 mini-shop 폴더에서 실행
편집기에서 JSX 작성 · 터미널에서 npm 명령 실행

브라우저 개발자 도구



Elements(요소)

현재 DOM 구조와 CSS 스타일 확인
글자·색상·배치가 이상할 때

Console(콘솔)

실행 중 발생한 오류와 로그 확인
클릭해도 동작하지 않을 때

Network(네트워크)

요청·응답과 상태 코드 확인
API 연결 시 활용

Windows Chrome · Ctrl+Shift+I 또는 우클릭 → 검사
Console(콘솔) 탭에서 오류 메시지와 요청 주소부터 확인

Prettier

정리 전

src/components/ProductCard.jsx

```
function ProductCard({product}) {  
  return <h3>{product.name}</h3>}
```

정리 대상

들여쓰기 · 공백 · 줄바꿈 · 따옴표 등 코드 형식

코드 형식 통일

붙여넣은 JSX의 가독성 개선과 형식 통일

실행 결과는 그대로

코드의 의미를 유지하며 형식 정리

ESLint는 설정된 규칙으로 일부 오류·문제 감지

Prettier로 정리 후

src/components/ProductCard.jsx

```
function ProductCard({ product }) {  
  return <h3>{product.name}</h3>;  
}
```

Prettier 설치와 자동 정리

01 VS Code 확장 설치

Ctrl+Shift+X → Prettier - Code formatter 검색

02 저장할 때 자동 정리

Ctrl+, → Default Formatter를 Prettier로 선택
Format On Save 체크

03 직접 정리하고 확인

JSX 파일에서 Shift+Alt+F로 문서 정리
들여쓰기를 흐트러뜨린 뒤 Ctrl+S로 정리되는지 확인

04 프로젝트 생성 후 버전 통일

오른쪽 명령으로 프로젝트에 Prettier 설치
설정 예시는 .vscode/settings.json에 저장

프로젝트 생성 후 · 터미널

터미널 · 프로젝트 폴더

```
npm install --save-dev --save-exact prettier
```

.vscode/settings.json

```
{
  "editor.defaultFormatter":
    "esbenp.prettier-vscode",
  "editor.formatOnSave": true
}
```

ESLint

src/App.jsx · 변수명 오류 예시

```
const price = 12000;  
console.log(pricee);
```

src/App.jsx · 수정 예시

```
const price = 12000;  
console.log(price);
```

예시 규칙 · no-undef

정의되지 않은 변수 pricee의 사용 감지

설정된 규칙에 따라 검사

미사용 변수 · React 작성 규칙 등 설정한 규칙 검사

도구별 역할

Prettier는 형식 정리 · ESLint는 규칙 검사 · Console은 실행 오류 확인

ESLint 설치와 검사 준비

01 VS Code 확장 설치

Ctrl+Shift+X → ESLint 검색 → 설치
(확장 ID: dbaeumer.vscode-eslint)

02 프로젝트 설정부터 확인

프로젝트 생성 후 mini-shop 폴더에서 진행
ESLint 패키지와 eslint.config.js 또는 .mjs가 있으면 기존 설정 사용

03 설정이 없을 때 설치·초기화

오른쪽 초기화 명령 실행 → JavaScript modules · React
TypeScript: No · Browser → 필요한 패키지 설치에 동의

04 검사 결과 읽기

npx eslint src로 검사하거나 Ctrl+Shift+M으로 Problems 확인
규칙 이름·파일·줄 번호를 기준으로 코드 수정

초기화와 검사 명령

터미널 · 프로젝트 폴더

```
npm init @eslint/config@latest  
npx eslint src
```

초기화 결과

프로젝트에 ESLint 패키지와
eslint.config.js 또는 .mjs 생성

역할 구분

VS Code 확장과 프로젝트 패키지·설정이 함께 필요
현재 예시 프로젝트는 Oxlint 사용
ESLint 실습은 위 절차로 별도 설정 후 진행

React Developer Tools

컴포넌트와 데이터를 직접 살펴보는 도구

Components

컴포넌트 구조와 Props·State 확인

Elements와의 차이

Elements: DOM·CSS

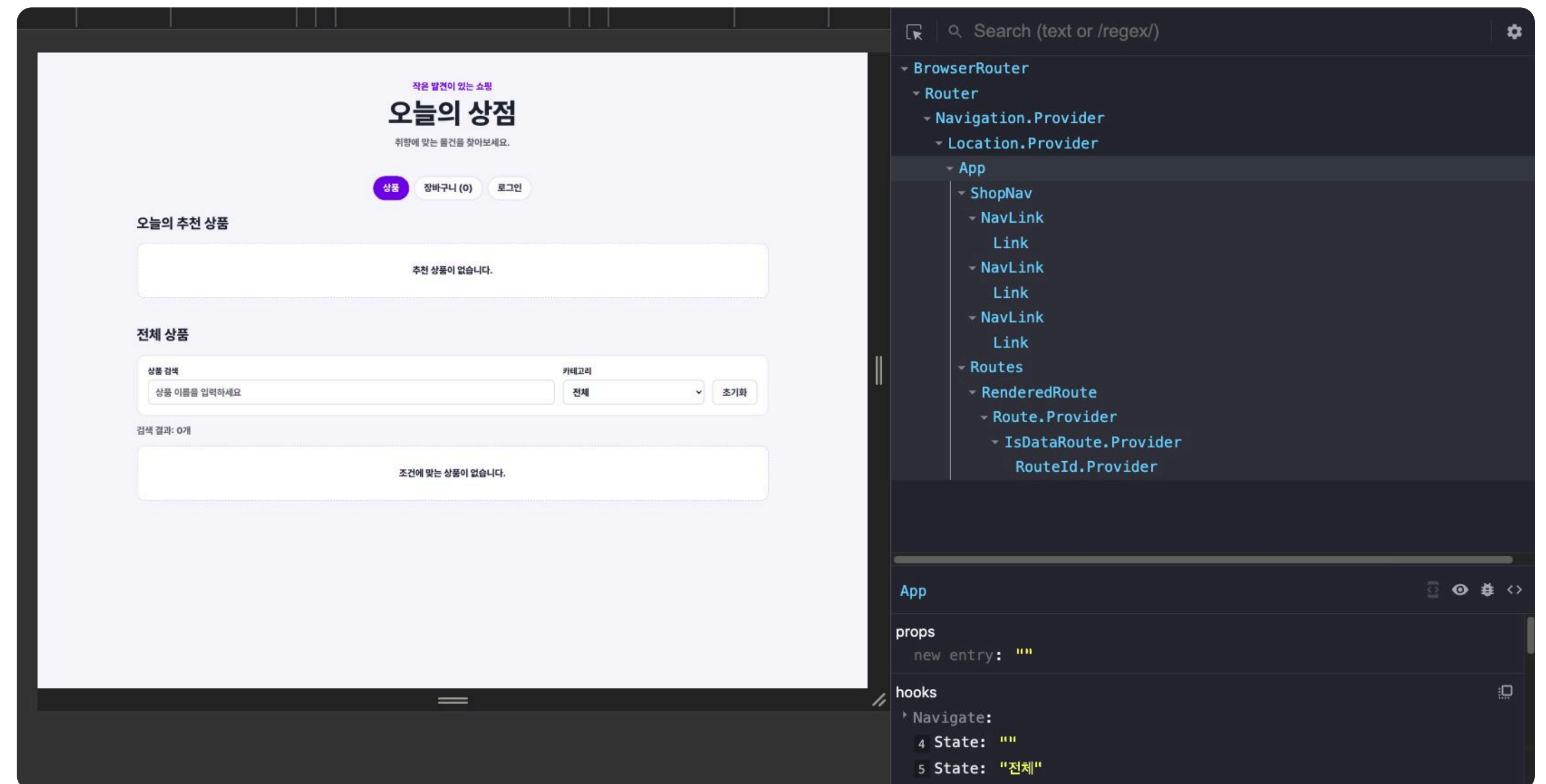
Components: React 컴포넌트 구조

Profiler

성능 분석 도구

이번 수업은 Components 중심으로 진행

Components 탭



React Developer Tools 설치

01 Chrome에서 공식 안내 열기

ko.react.dev/learn/react-developer-tools

02 확장 프로그램 추가

공식 안내의 Chrome 링크 → Chrome에 추가 → 확장 프로그램 추가

03 React 앱에서 탭 확인

앱 실행 후 Ctrl+Shift+I → Components 탭에서 App 확인

04 설치와 실제 확인을 구분

준비 단계에서 확장 설치

앱 생성 후 Components·Profiler 확인

React 소개와 개발 준비 · 핵심 정리

React의 역할

UI를 만드는 JavaScript 라이브러리
HTML·CSS·JavaScript 기초 위에서 활용
화면 변화가 많을수록 상태 관리에 도움

실행·개발 환경

Node.js: JS 런타임 · npm: 패키지 관리
Vite: 개발 서버·빌드
VS Code: 코드 작성·터미널 실행

화면 구성과 동작

컴포넌트 조합·재사용과 State에 맞는 UI 선언
DOM: 문서 객체 · SPA: 같은 문서에서 전환
CSR: 브라우저에서 UI 생성

코드 품질 도구

Prettier: 들여쓰기·줄바꿈 등 형식 정리
ESLint: 설정된 규칙에 따라 코드 검사

State와 단방향 데이터 흐름

사용자 입력 → State 변경 → 화면 반영
Props는 부모에서 자식으로 전달
변경은 State를 가진 컴포넌트가 처리

화면·데이터 확인 도구

브라우저: DOM·CSS·오류·요청 확인
React Developer Tools: 컴포넌트 확인
Components에서 Props·State 확인

2교시

JSX로 상점 첫 화면 만들기

미션 - 브라우저에서 내 상점 실행

배울 내용

Vite 실행 · 파일 구조 · 컴포넌트 · JSX

완성 기준

상점 이름 · 상품 이름 · 가격 · 버튼 표시

프로젝트 실행하기

설치 확인

Node.js와 npm 설치 확인

프로젝트 생성과 실행

명령을 한 줄씩 실행

브라우저 접속

터미널에 표시된 로컬 주소로 접속

터미널 · 프로젝트 생성과 실행

```
node -v
npm -v

npm create vite@latest mini-shop -- --template react
cd mini-shop
npm install
npm run dev
```

첫 실행 확인

01 첫 화면 확인

터미널의 주소로 접속
Vite 시작 화면 표시 확인

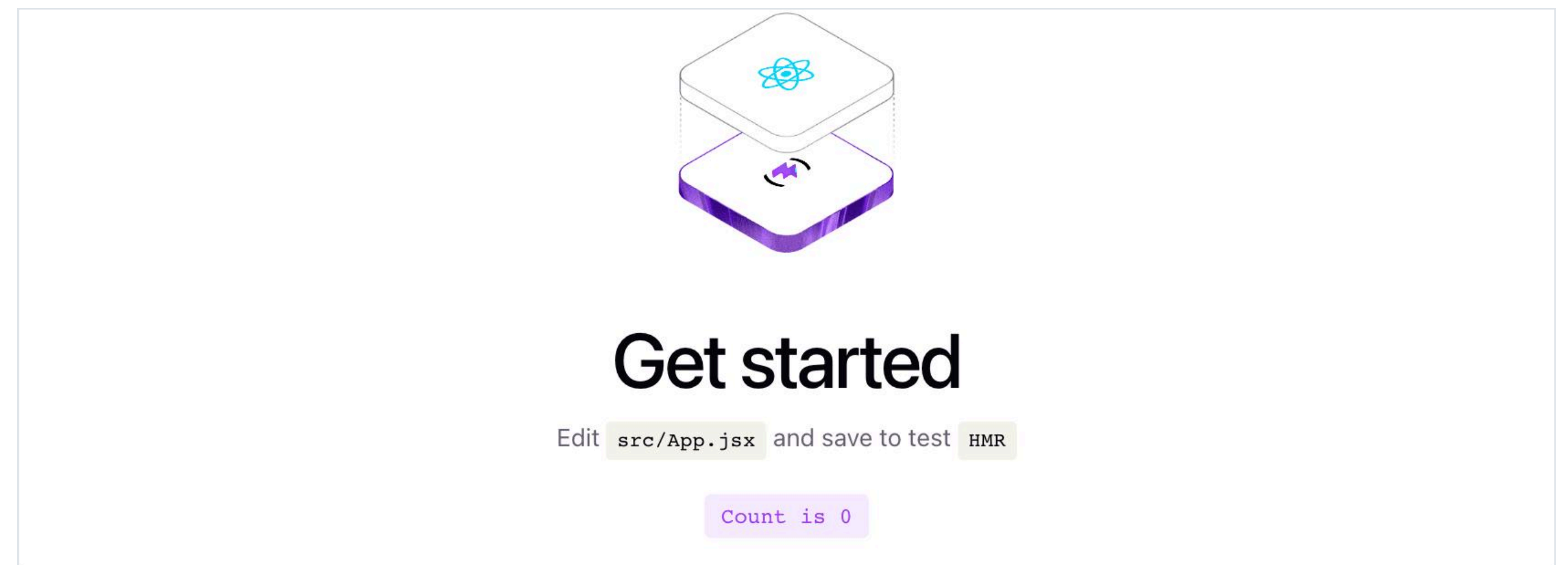
02 App 선택

우클릭 → 검사 → Components
컴포넌트 트리에서 App 선택

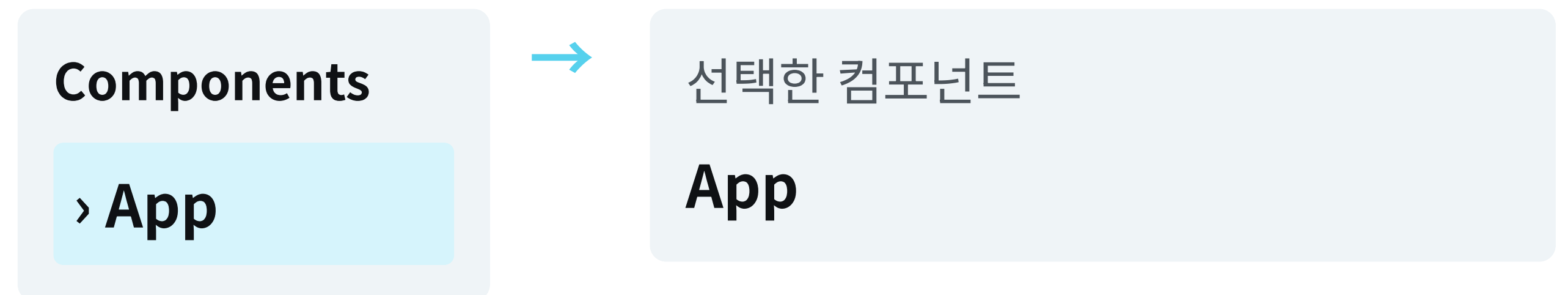
03 탭 확인

React Developer Tools 활성화
앱 실행 확인 후 새로고침

Vite 첫 실행 화면



Components 확인 예시



첫 화면을 구성하는 세 파일

확장된 원예제의 주요 파일 · 수업 진행 중 순차 추가

mini-shop/

- **index.html**
- src/
- public/
- package.json
- └ vite.config.js



index.html

브라우저가 처음 읽는
HTML 파일

src/

- **main.jsx**
- **App.jsx**
- index.css
- App.css
- assets/
- └ components/



src/main.jsx

React를 시작하고
App을 화면에 연결

src/components/

- ProductCard.jsx · 3교시 추가
- └ ShopNav.jsx · 후속 수업



src/App.jsx

상점 화면의 내용 작성

JSX로 화면 표현

JSX(JavaScript XML)

JavaScript 안에서 HTML과 비슷한 문법으로
화면을 표현하는 문법 확장임

src/App.jsx · 컴포넌트 · JSX 일부 발췌

```
const shopName = "오늘의 상점";  
  
<h1>{shopName}</h1>
```

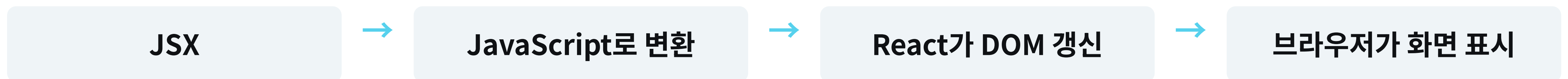
태그: 화면 구조

<h1>은 화면의 제목을 표현

중괄호: JavaScript 값 표시

{shopName}에 변수의 값 삽입

JSX 변환과 화면 표시



시맨틱 태그로 화면 구조 표현

각 태그로 콘텐츠의 역할 표현 · 실제 배치는 CSS로 지정

src/App.jsx

```
<main>
  <header><h1>오늘의 상점</h1></header>
  <section>
    <h2>오늘의 추천 상품</h2>
    <article>
      <h3>캔버스 토트백</h3>
      <p>18,000원</p>
    </article>
  </section>
</main>
```

코드 구조 이해 · 보조기기의 탐색 지원 · 콘텐츠 역할 전달

표시되는 main은 하나 · section은 일반적으로 제목과 함께 사용
메뉴는 nav, 하단 정보는 footer.

main	페이지의 핵심 내용
header	영역의 소개와 제목
section	같은 주제로 묶은 영역
article	독립적으로 이해할 수 있는 내용

h1 → h2 → h3: 제목의 관계 표현

JSX의 기본 규칙

하나의 루트 요소

여러 요소를 하나의 태그 또는
Fragment(<>)로 감싸기

태그 닫기

여는 태그와 닫는 태그의 짝 맞추기
자식이 없는 태그는 형태로 작성

className

CSS 클래스는 className으로
지정

JavaScript 표현식

{shopName}처럼 변수나
계산 결과를 JSX에 삽입

src/App.jsx

```
function ShopTitle() {  
  const shopName = "오늘의 상점";  
  return (  
    <header className="shop-header">  
      <h1>{shopName}</h1>  
      <p>작은 발견이 있는 쇼핑</p>  
    </header>  
  );  
}
```

컴포넌트 작명과 함수 작성 규칙

컨벤션

함께 읽고 관리하기 위한 코드 작성 약속

컴포넌트와 파일: PascalCase

ShopTitle · ShopTitle.jsx

컴포넌트 이름은 대문자로 시작

변수와 일반 함수: camelCase

shopName · formatPrice

boolean: is · has 접두어

isOpen: 열림 상태

isSoldOut: 품절 여부

hasItems: 상품 존재 여부

is: 상태 · has: 보유 · 존재

function으로 작성

src/App.jsx

```
function ShopTitle() {  
  return <h1>오늘의 상점</h1>;  
}
```

화살표 함수로 작성

src/App.jsx

```
const ShopTitle = () => {  
  return <h1>오늘의 상점</h1>;  
};
```

function · 화살표 함수 모두 사용 가능

교안 컨벤션: 컴포넌트는 function

짧은 콜백은 화살표 함수

상점 첫 화면 코드

src/App.jsx · 앞부분

```
function App() {
  const shopName = "오늘의 상점";
  const productName = "캔버스 토트백";
  const price = 18000;
  return (
    <main>
      <header>
        <p>작은 발견이 있는 쇼핑</p>
        <h1>{shopName}</h1>
      </header>
      <section>
        <h2>오늘의 추천 상품</h2>
        <article>
```

src/App.jsx · 이어지는 부분

```
        <div
          aria-label="토트백 상품 이미지"
          role="img"
        >
          
        </div>
        <h3>{productName}</h3>
        <p>{price.toLocaleString()}원</p>
        <button type="button">
          상품 보러 가기
        </button>
      </article>
    </section>
  </main>
);
}
export default App;
```

코드와 렌더링 결과의 연결

코드에 적은 값과 표현식

상점 이름

```
shopName = "오늘의 상점"  
<h1>{shopName}</h1>
```

상품 이름

```
productName = "캔버스 토트백"  
<h3>{productName}</h3>
```

가격

```
price = 18000  
{price.toLocaleString()}원
```

렌더링 실제 화면



실습 - 나만의 상점

01 상점 문구 변경

상점 이름과 소개 문구를 내 콘셉트로 변경

02 상품 설정

상품 이름과 숫자 가격, 버튼 문구 변경

03 실행 결과 확인

저장 후 브라우저가 바뀌는지 확인

2교시 핵심 정리

컴포넌트

이 수업에서는 JSX로 UI를 표현하는 JavaScript 함수로 작성

JSX와 JavaScript 표현식

하나의 루트 요소로 감싸고, {}에 변수·계산 결과를 넣어 표시

시맨틱 태그

main·header·section·article로 콘텐츠의 역할을 표현

파일과 화면 연결

index.html: 브라우저가 처음 읽는 파일

main.jsx: App을 화면에 연결 · App.jsx: 화면 내용 작성

3교시

컴포넌트로 상품 목록 만들기

미션 - 하나의 카드 구조로 여러 상품 표시

배울 내용

컴포넌트 재사용 · Props · map · key

완성 기준

상품 데이터 추가 시 카드도 자동 추가

상품 정보를 배열로 준비

상품 하나 = 객체

id, name, category, price, emoji

여러 상품 = 배열

상품 데이터를 하나의 배열로 관리

가격은 숫자

계산과 숫자 표시를 위해 가격은 숫자로 저장

src/App.jsx

```
const products = [
  { id: 1, name: "캔버스 토트백", category: "가방",
    price: 18000, emoji: "👜" },
  { id: 2, name: "데일리 머그컵", category: "주방",
    price: 12000, emoji: "☕" },
  { id: 3, name: "포켓 노트", category: "문구",
    price: 4500, emoji: "📓" },
];
```

객체(Object) - 관련 값을 속성 이름과 함께 묶은 데이터

{ name: "캔버스 토트백", price: 18000 }에서 name·price는 속성, 뒤의 값은 속성값임
product.name으로 이름 읽기 products 배열에 여러 상품 객체 저장

Props의 개념

Props(Properties) - 부모가 자식 컴포넌트에 전달하는 입력값

문자열 · 숫자뿐 아니라 객체 · 배열 · 함수도 전달 가능
자식은 전달받은 Props를 읽기 전용으로 사용

src/App.jsx

```
<ProductCard product={products[0]} />  
<ProductCard product={products[1]} />
```

같은 구조, 다른 상품 데이터

ProductCard는 카드의 공통 모양 담당
서로 다른 product 값으로 여러 상품 표시

읽기 전용

값의 변경은 데이터를 관리하는 부모에서 처리

ProductCard로 공통 구조 재사용

같은 카드 구조 재사용

상품마다 태그를 다시 쓰지 않고
ProductCard를 여러 번 사용

상품 데이터 받기

{ product }는 Props 객체에서
product 속성을 꺼내는 구조 분해 할당

값만 바꾸어 표시

같은 구조에 서로 다른 상품의
카테고리·이름·가격 전달

src/components/ProductCard.jsx

```
function ProductCard({ product }) {  
  return (  
    <article className="product-card">  
      <p>{product.category}</p>  
      <h3>{product.name}</h3>  
      <p>{product.price.toLocaleString()}원</p>  
      <button type="button">상품 자세히 보기</button>  
    </article>  
  );  
}  
export default ProductCard;
```

ProductCard 한 번 작성 + 서로 다른 product 전달 = 여러 상품 카드

Props의 전달 방향

Props는 부모 컴포넌트에서 자식 컴포넌트로 전달

src/App.jsx

```
// 부모의 JSX 내부  
<ProductCard product={products[0]} />
```

src/components/ProductCard.jsx

```
function ProductCard({ product }) {  
  return <h3>{product.name}</h3>;  
}
```

부모 App의 상품 객체를 자식 ProductCard에서 읽기

Props는 읽기 전용임 · 자식에서 product.name 직접 변경 금지

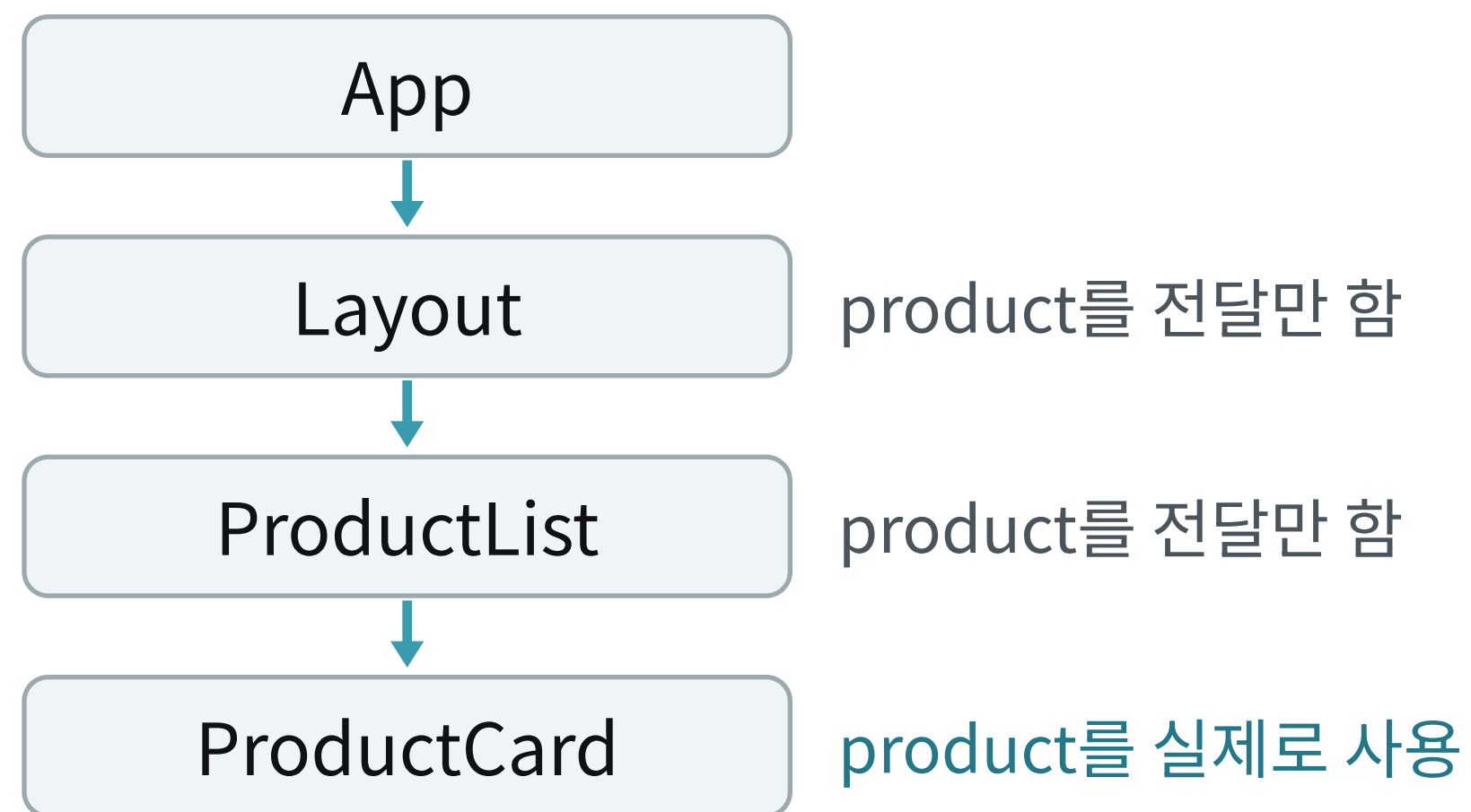
자식의 이벤트로 부모 상태를 바꿀 때

부모가 함수를 Props로 전달 → 자식이 함수 호출 → 부모가 상태 갱신
공유 범위가 넓어지면 Context로 전달 · 복잡한 상태 관리에는 Redux 등 검토

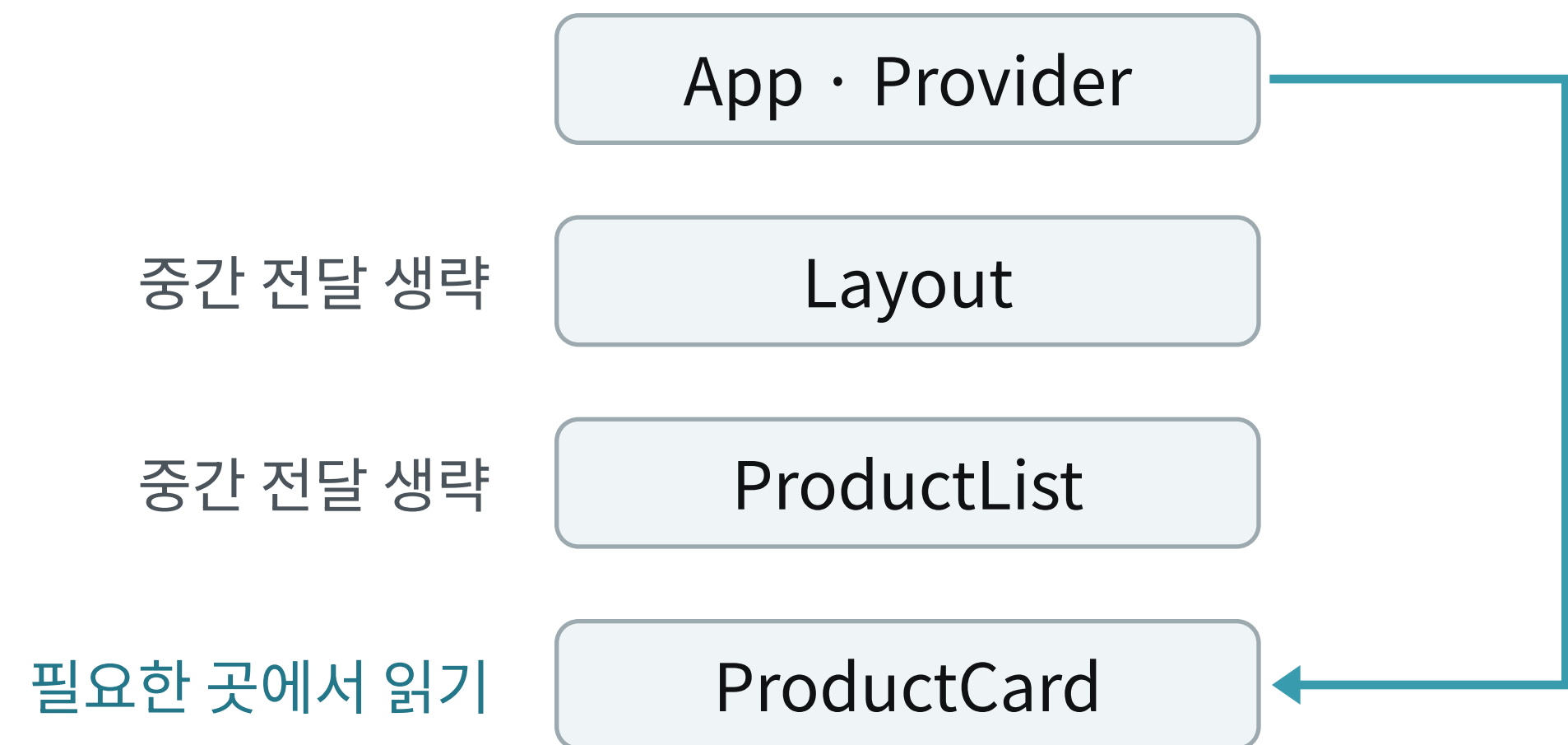
Props drilling과 상태 공유

깊은 자식에게 값을 전달하려고 중간 컴포넌트가 Props를 계속 전달하는 상황

Props drilling



Context API



TIP - 구조를 먼저 단순하게

필요한 곳 가까이에 상태 배치.

컴포넌트 조합이나 children 사용도 검토.

현재 App과 ProductCard의 직접 연결에는 Props만으로 충분함

공유 범위가 넓어질 때

Context API로 하위 트리에 데이터 공유.

Redux는 공유 상태와 변경 규칙을 관리하는 라이브러리.

JavaScript map 함수

배열의 각 항목을 변환하고, 결과를 새 배열로 반환하는 함수

src/App.jsx · map 설명 예시

```
const names = products.map((product, index) => {  
  return `${index + 1}. ${product.name}`;  
});
```

콜백의 매개변수 순서

(항목, index, 원본 배열)

첫 번째는 product, 두 번째가 index.

index는 0부터 시작하는 현재 위치.

변환 결과

["1. 캔버스 토트백", "2. 데일리 머그컵", "3. 포켓 노트"]

리스트 렌더링에 연결

이름 대신 JSX를 반환하면 상품 카드 JSX 배열을 생성 가능

map 자체는 원본 배열을 변경하지 않음 · 콜백에서 상품 객체 직접 변경 금지

리스트 렌더링

상품 객체를 카드로 변환

map의 콜백이 ProductCard JSX 반환

key로 항목 식별

같은 부모 아래의 목록 항목을
고유하고 안정적인 값으로 구분

상품 ID 사용

추가·삭제·정렬 뒤에도
같은 상품을 같은 id로 식별

src/App.jsx

```
import ProductCard from "../components/ProductCard";

<div className="product-grid">
  {products.map((product) => (
    <ProductCard
      key={product.id}
      product={product}
    />
  ))}
</div>
```

index를 key로 쓸 때의 문제

index는 현재 위치 → 항목을 삭제하거나 정렬하면 같은 key가 다른 상품을 가리킬 수 있음

삭제 전 - 토트백 메모에 “선물용” 입력

key 0 토트백 메모: 선물용
key 1 머그컵 메모: 빈 값

src/App.jsx · map 내부의 index key 예시

```
<ProductCard  
  key={index} product={product}  
/>
```

토트백 삭제 후 - index를 key로 사용

key 0 머그컵 메모: 선물용
같은 위치의 입력값 · 컴포넌트 상태를 잘못 재사용할 수 있음

src/App.jsx · 권장 예시

```
<ProductCard  
  key={product.id} product={product}  
/>
```

안정적인 ID로 상품과 상태 연결

id 2는 삭제 · 정렬 뒤에도 머그컵임 · 입력값 · 내부 State를 같은 상품에 연결
위 메모 입력란은 설명용 추가 예시임 · 항상 실행 오류가 발생하는 것은 아님

상품 목록과 스타일 파일

원예제의 주요 파일 · ProductCard는 3교시, ShopNav는 후속 수업

mini-shop/

- index.html
- src/
- public/
- package.json
- └ vite.config.js



src/App.jsx

ProductCard를 불러와
상품 목록 표시

src/

- main.jsx
- **App.jsx**
- **index.css**
- **App.css**
- assets/
- └ components/



src/App.css

상품 그리드 · 카드 · 버튼의
모양 정의

src/components/

- ProductCard.jsx · 3교시 추가
- └ ShopNav.jsx · 후속 수업



src/index.css

여백 · 글꼴 · 배경 등
페이지 공통 스타일

실습 - 상품 추가

01 새 상품 객체 추가

기존 상품과 다른 id 지정

02 정보 입력

name · category · price · emoji 입력

03 화면과 Props 확인

새 카드 확인 → Components에서 ProductCard 선택 → product의 이름 · 가격 비교

3교시 핵심 정리

상품 데이터

상품 하나는 객체, 여러 상품은 배열.
속성 이름으로 필요한 값 읽기

Props drilling

중간 컴포넌트의 연속 전달
구조 단순화 후 Context·상태 관리 도구 검토

컴포넌트 재사용

ProductCard에 공통 카드 구조 작성.
서로 다른 상품 데이터 전달

map과 리스트 렌더링

각 상품을 JSX로 변환해 새 배열 반환.
콜백의 인자 순서는 (항목, index).

Props와 전달 방향

부모가 자식에 전달하는 읽기 전용 값.
자식의 변경 요청은 전달받은 함수 호출.

안정적인 key

항목의 고유 ID로 식별.
삭제·정렬 뒤에도 상품과 상태 연결.

실습 점검

import 경로·대소문자, export, 중복 id·key, 가격의 숫자 타입 확인.
Components 탭의 product 값과 카드의 이름·가격 비교.

4교시

State로 검색과 필터 구현하기

미션 - 검색 조건에 맞는 상품만 표시

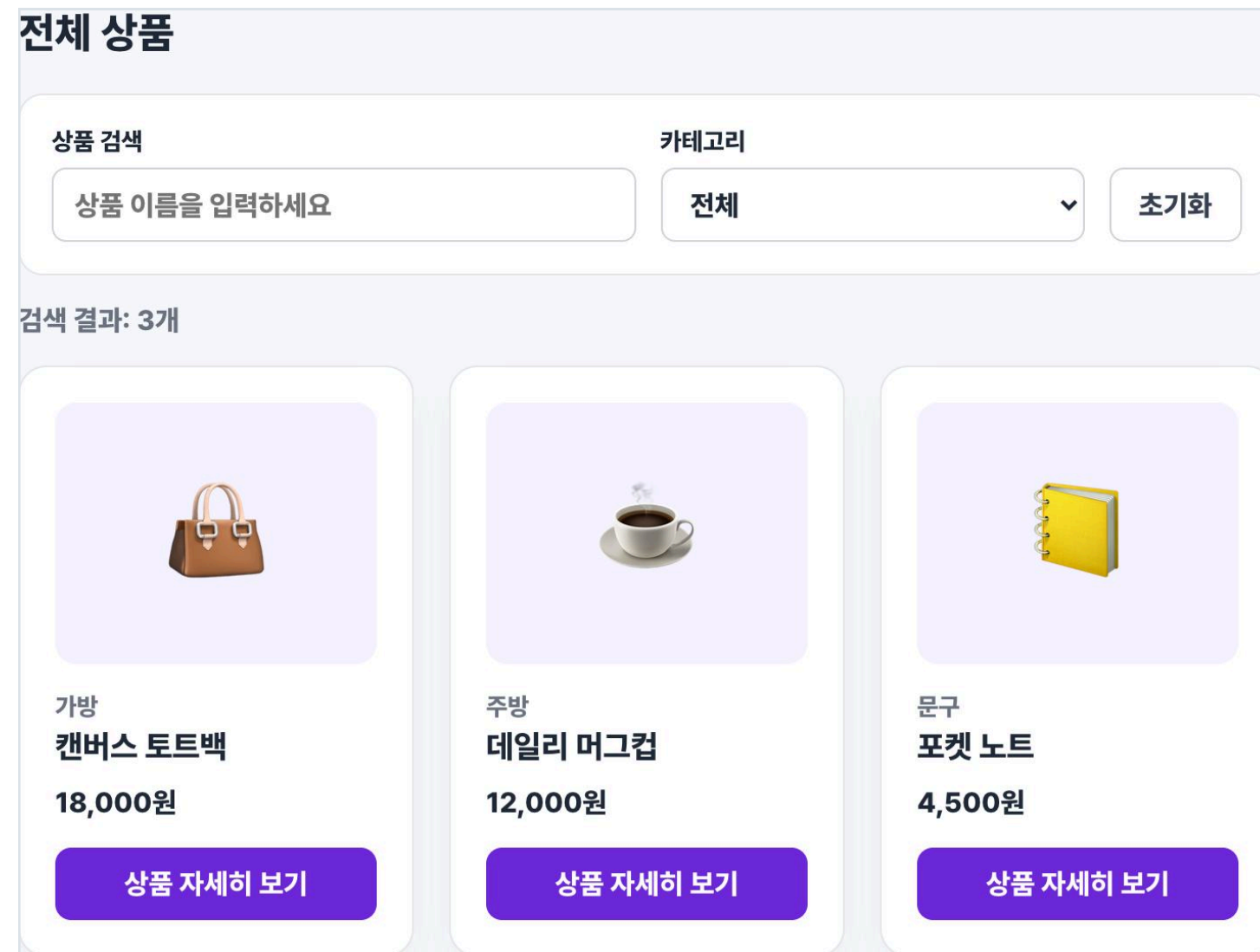
배울 내용

이벤트 · useState · 제어 입력 · filter · 조건부 렌더링

완성 기준

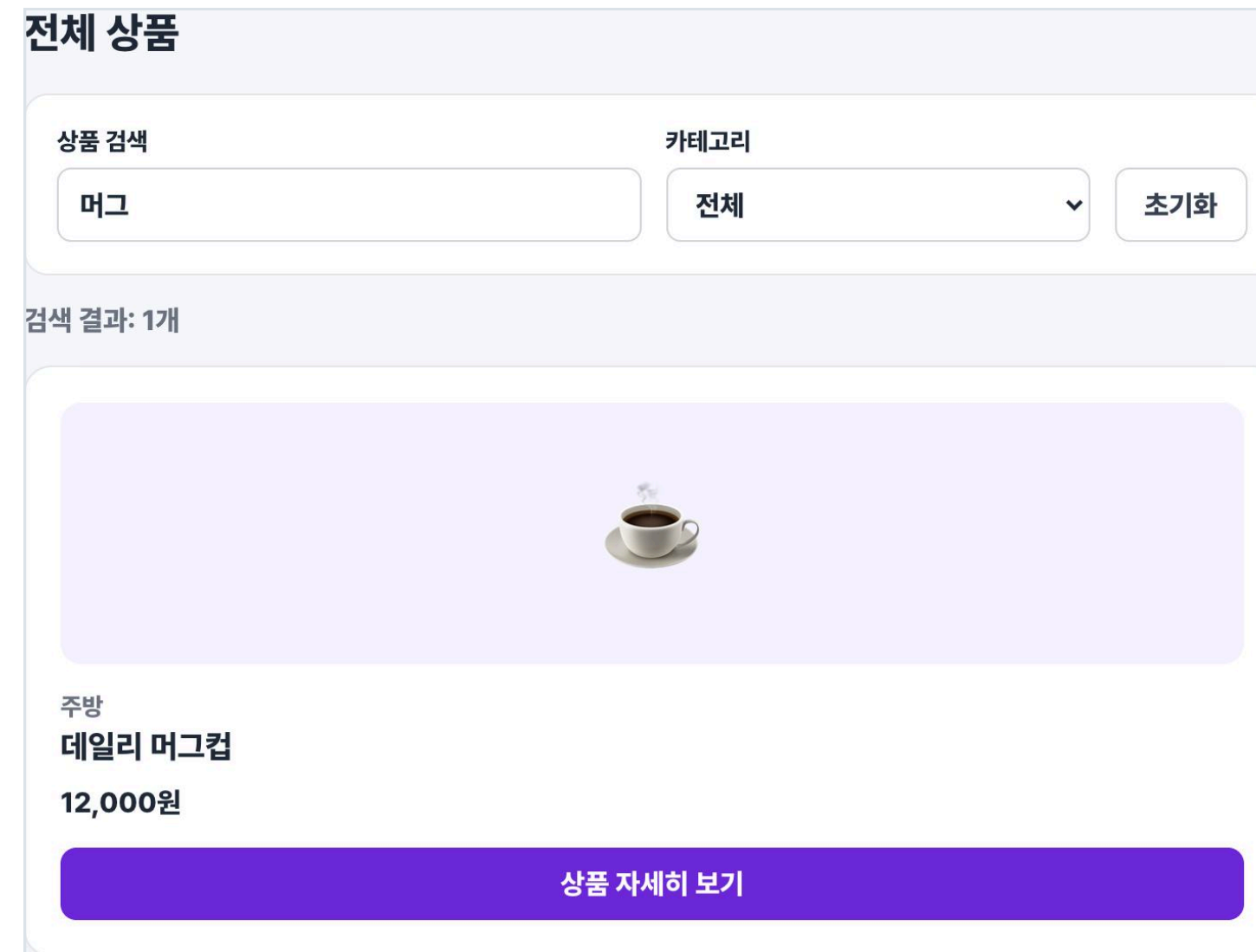
검색 · 카테고리 · 결과 수 · 빈 결과 · 초기화 동작

검색 조건별 렌더링 결과



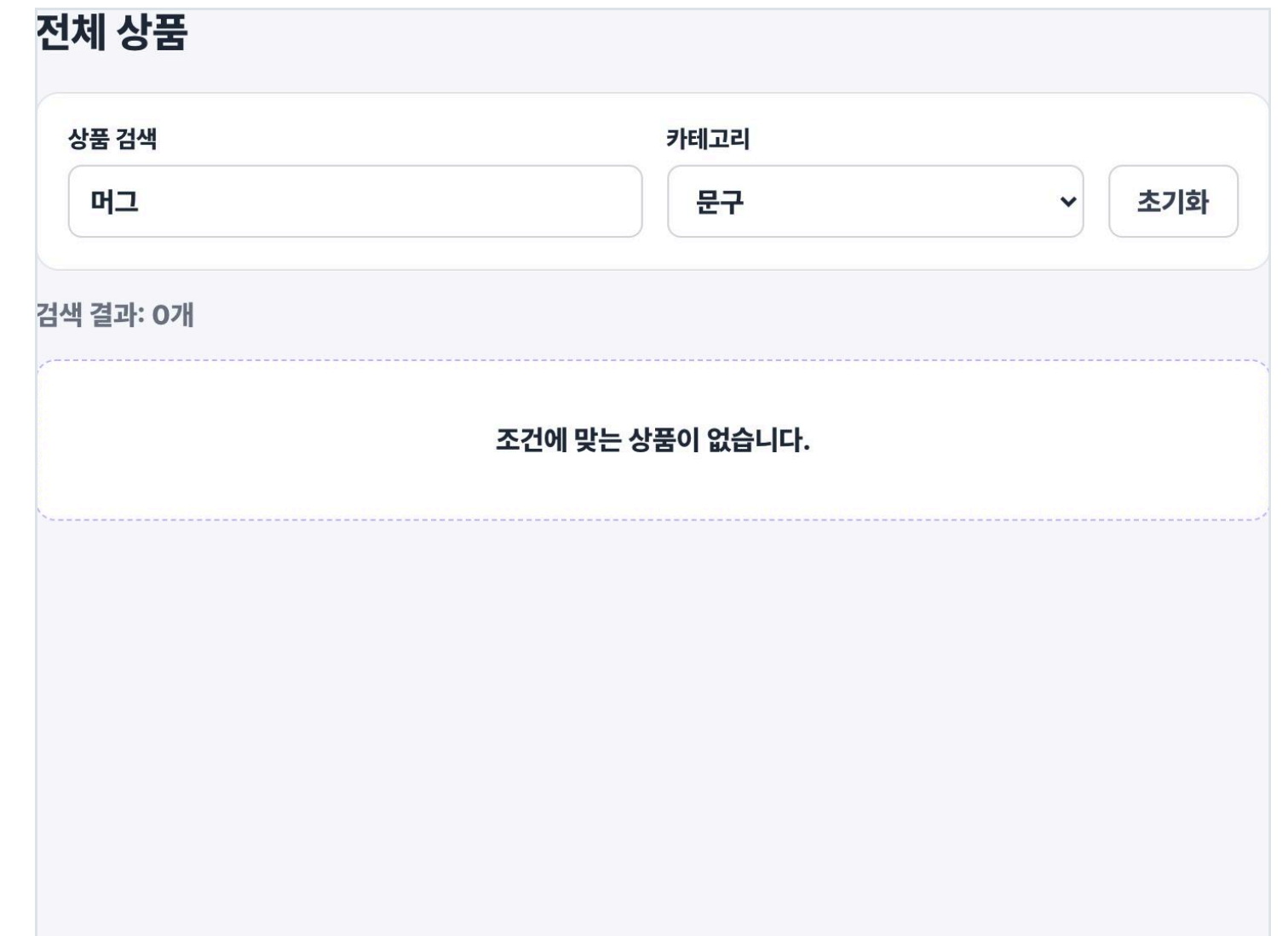
검색어 없음
카테고리 전체

상품 3개와
검색 결과 3개 표시



검색어 머그
카테고리 전체

머그컵 카드 1개와
검색 결과 1개 표시



검색어 머그
카테고리 문구

검색 결과 0개와
빈 결과 안내 표시

일반 변수와 State의 차이

일반 변수

일반 변수의 값 변경은 React에 화면 갱신을 요청하지 않음

컴포넌트가 다시 실행되면

예시의 `let count = 0`은 컴포넌트가 다시 실행될 때 0으로 초기화됨

useState

렌더링 사이에 값을 유지하고 setter로 갱신 요청

src/App.jsx · 비교용 컴포넌트 예시

```
import { useState } from "react";

// 일반 변수
function PlainCounter() {
  let count = 0;
  return <button onClick={() => count++}>{count}</button>;
}

// State
function StateCounter() {
  const [count, setCount] = useState(0);
  return <button onClick={() => setCount(c => c + 1)}>
    {count}
  </button>;
}
```

useState

src/App.jsx · useState import 후 App 함수 최상위

```
const [count, setCount] = useState(0);
```

count: 현재 값

첫 렌더링에서 0
이후에는 React가 기억한 값

setCount: 갱신 함수

setCount(1)로 다음 값 요청
이전 값 기준: $prev \Rightarrow prev + 1$

useState(0): 초기값

0은 처음 상태를 정하는 값
리렌더링마다 0으로 덮지 않음

배열 구조 분해 할당 (디스트럭처링)

ES6 문법의 활용
[현재 값, 갱신 함수] 배열에서
두 항목을 순서대로 꺼내 이름을 붙임

src/App.jsx · App 내부의 대체 표현

```
const state = useState(0);  
const count = state[0];  
const setCount = state[1];
```

useState는 컴포넌트 · 커스텀 Hook의 최상위에서 호출 · 조건문 · 반복문 · 이벤트 함수 안 호출 금지

입력값을 상태로 기억

초기값

검색어는 빈 문자열, 카테고리는 전체

value

입력창에 현재 State 표시

onChange

현재 입력값을 setter에 전달
다음 렌더링의 State와 화면 갱신

State와 갱신 함수 이름

searchTerm / setSearchTerm
갱신 함수 이름은 set + State 이름

src/App.jsx

```
import { useState } from "react";

function App() {
  const [searchTerm, setSearchTerm] = useState("");
  const [selectedCategory, setSelectedCategory] =
    useState("전체");
  // 아래는 검색창 부분만 발췌
  return (
    <input value={searchTerm}
      onChange={(event) =>
        setSearchTerm(event.target.value)}
    />
  );
}
```

입력에서 화면까지의 흐름

01 ① 사용자가 입력

검색창에 머그를 입력

02 ② 이벤트 처리

onChange → event.target.value → setSearchTerm

03 ③ 다시 계산

React가 App을 다시 실행해 필터 결과 계산

04 ④ 화면 반영

머그컵 카드와 검색 결과 1개 표시

리렌더링과 브라우저 새로고침

State 갱신과 리렌더링

setter가 다음 State와 화면 갱신을 요청
React가 컴포넌트를 실행해 화면 계산
필요한 변경을 DOM에 반영
같은 위치 · 타입 · key의 컴포넌트는 State 유지
검색 입력은 페이지를 다시 로드하지 않음

브라우저 새로고침

페이지를 다시 로드하고 앱을 새로 시작
이 앱의 State는 메모리에만 저장
저장한 검색어와 선택값이 사라지고
useState의 초기값으로 다시 시작

머그 + 문구에서 새로고침한 뒤

searchTerm: "머그" → "" selectedCategory: "문구" → "전체"

조건 초기화 → filteredProducts 재계산 → 결과 0개에서 3개로 변경

새로고침 뒤에도 값을 보관하는 방법

localStorage

같은 출처(origin)에서 브라우저 재시작 후에도 보관
사용자 삭제 · 브라우저 설정 등에 따라 삭제 가능

테마 · 검색 조건 등 작은 설정

sessionStorage

같은 출처 · 탭 세션에서 새로고침 후 유지
탭 세션 종료 시 삭제

한 탭에서 이어갈 입력값

서버와 DB

서버에 저장하고 API로 다시 조회
계정과 연결하면 다른 기기에서도 조회

주문 · 상품 · 장바구니 데이터

다음 수업: React와 FastAPI · PostgreSQL 연결



저장한 데이터는 조회 응답으로 받아 화면의 State에 반영

조건부 렌더링: 조건에 따라 화면 선택

JSX 안에서 JavaScript 조건식으로 표시할 요소 선택

삼항 연산자: 둘 중 하나 표시

src/App.jsx · 구조 설명 예시

```
{filteredProducts.length > 0  
  ? <p>상품 목록 표시</p>  
  : <p>조건에 맞는 상품이 없습니다.</p>}
```

조건이 참이면 ? 뒤, 거짓이면 : 뒤의 JSX 선택

&&: 조건이 참일 때만 표시

src/App.jsx · 구조 설명 예시

```
{filteredProducts.length === 0 && (  
  <p>조건에 맞는 상품이 없습니다.</p>  
)}
```

조건이 거짓이면 해당 안내를 렌더링하지 않음

숫자 0을 && 왼쪽에 두면 0이 화면에 표시될 수 있음

length && <p>...</p> 대신 length > 0 && <p>...</p>처럼 boolean 조건 사용

filter로 상품 선택 → 결과 유무에 따라 상품 카드 또는 안내 표시

TIP: &&와 ||의 단축 평가

왼쪽부터 평가함 · 왼쪽에서 결과가 결정되면 오른쪽 식 평가 생략

A && B

A가 falsy이면 A를 반환, B는 평가 생략
A가 truthy이면 B를 평가하고 반환

src/App.jsx · 평가 설명 예시

```
false && check() // check 실행 안 함  
true && check()  // check 실행
```

A || B

A가 truthy이면 A를 반환, B는 평가 생략
A가 falsy이면 B를 평가하고 반환

src/App.jsx · 평가 설명 예시

```
true || check() // check 실행 안 함  
false || check() // check 실행
```

카테고리가 전체이면 오른쪽 비교 생략

```
selectedCategory === "전체" || product.category === selectedCategory
```

truthy / falsy: 조건에서 참·거짓으로 해석되는 값. falsy 예: false, 0, "", null, undefined, NaN
&&·||는 피연산자 값을 반환함 · 이번 실습의 비교식 결과는 boolean임

두 조건을 함께 적용

이름 검색

앞뒤 공백 제거와 대소문자 통일

카테고리

전체 또는 선택한 카테고리와의 일치

&&

두 조건을 모두 만족하는 상품만 표시

src/App.jsx

```
const filteredProducts = products.filter((product) => {  
  const matchesSearch = product.name  
    .toLowerCase()  
    .includes(searchTerm.trim().toLowerCase());  
  
  const matchesCategory =  
    selectedCategory === "전체" ||  
    product.category === selectedCategory;  
  
  return matchesSearch && matchesCategory;  
});
```

결과 수와 빈 화면 안내

결과 수

filteredProducts.length

결과가 있으면

filteredProducts.map으로 카드 표시

결과가 없으면

조건에 맞는 상품이 없다는 안내 표시

src/App.jsx

```
<p>검색 결과: {filteredProducts.length}개</p>

{filteredProducts.length > 0 ? (
  <div className="product-grid">
    {filteredProducts.map((product) => (
      <ProductCard key={product.id} product={product} />
    ))}
  </div>
) : (
  <p className="empty-state">조건에 맞는 상품이 없습니다.</p>
)}
```

TIP: 프로퍼티와 메서드

프로퍼티: 객체의 속성 · 배열도 객체임

.length는 배열의 길이 · 이 예제에서는 상품 수와 같음

메서드: 함수인 프로퍼티 · .map(...)은 ()로 호출함

카테고리 선택과 초기화

select도 제어 입력

value와 onChange 연결

선택값 갱신

선택한 카테고리를 State에 저장

초기화

검색어와 카테고리를 초기값으로 복원
앱을 다시 시작하지 않고 State만 변경

src/App.jsx

```
// App 함수 내부: 초기화 핸들러
function resetFilters() {
  setSearchTerm("");
  setSelectedCategory("전체");
}

// App의 return 내부: JSX 일부 발췌
<select value={selectedCategory}
  onChange={(e) => setSelectedCategory(e.target.value)}>
  {categories.map((category) => (
    <option key={category} value={category}>
      {category}
    </option>
  ))}
</select>
<button type="button" onClick={resetFilters}>초기화</button>
```

상태와 계산값을 구분

State는 입력·선택값을 렌더링 사이에 기억함 · 계산값은 현재 데이터와 조건에서 도출

상품 원본

products: 전체 3개

기억해야 하는 State

searchTerm = "머그"

selectedCategory = "문구"

이전 입력에서 무엇을 선택했는지

현재 상품 배열만으로는 알 수 없음

현재 조건으로 계산

src/App.jsx · 계산 흐름 요약

```
const filteredProducts =  
  products.filter(...);
```

입력 조건이 바뀔 때 다시 계산

원본 products는 그대로 유지

계산한 결과

filteredProducts = []

filteredProducts.length = 0

조건부 렌더링

검색 결과: 0개

조건에 맞는 상품이 없습니다.

결과와 결과 수를 별도 State에 넣으면 검색 조건과 결과가 어긋날 수 있어 렌더링 중에 계산.

실습 - React Developer Tools에서 State 확인

01 React Developer Tools의 Components 탭

브라우저 개발자 도구를 열고, 설치한 React Developer Tools의 Components 탭 선택
React 앱이 실행된 페이지에서 App 컴포넌트 선택

02 Hooks / State 값과 화면 비교

검색창에 머그 입력: 해당 State 값 "머그", 화면에는 머그컵 카드 1개
카테고리 문구 선택: 해당 State 값 "문구", 검색 결과는 0개

03 초기화 버튼과 새로고침 확인

두 State 값이 ""와 "전체"로 돌아오고, 화면에 상품 3개 표시
filteredProducts와 결과 수는 계산값이므로 별도 State 항목으로 저장하지 않음

Elements에서 DOM 확인 · React Developer Tools에서 React의 Props · State 확인

실습 - 검색 시나리오 검증

01 화면과 State 연결

전체 3개 → 머그 입력 → State 값 머그, 머그컵 1개

02 조건 조합

머그 + 문구 선택 → 결과 0개와 안내 문구

03 초기화와 공백

초기화 → 3개 / 검색어 앞뒤에 공백을 넣은 ‘ 토트백 ’ → 1개

04 새로고침

머그 + 문구에서 새로고침: 검색어 빈 값, 카테고리 전체, 상품 3개

4교시 핵심 정리와 오류 점검

useState와 입력값

[현재 값, setter]를 배열 구조 분해 할당으로 받음
value에 State 표시, onChange에서 setter 호출

계산과 조건부 렌더링

filter로 이름 · 카테고리 조건을 모두 검사
결과 수는 length, 화면 선택은 ? : 또는 &&

새로고침과 저장

리렌더링 사이에는 State 유지
새로고침 뒤 유지하려면 별도 저장 · 복원 필요

확인 도구: React Developer Tools의 Components에서 App의 Hooks / State 확인

검색 결과의 중복 State 저장 불필요 · 이번 필터 계산에는 useEffect 불필요

파일과 오류 확인

App.jsx에 단계별 추가 · 전체 코드는 복구용
App.css에 필터 스타일 추가
useState import · 최상위 호출 확인
value / onChange 연결 확인
filteredProducts.map으로 결과 표시
상품 id를 key로 사용
length > 0으로 boolean 조건 검사

1일차 데이터 정리

원본 데이터와 State

products: 상품 원본 배열

searchTerm: 입력한 검색어

selectedCategory: 선택한 카테고리

두 검색 조건은 렌더링 사이에 기억하는 State

렌더링할 때 계산하는 값

filteredProducts: 조건에 맞는 상품

filteredProducts.length: 검색 결과 수

현재 상품과 조건에서 매번 계산

필터 결과의 중복 State 저장 불필요

1일차 완성 - 다음 날 미션

상품 목록 구성과 검색 · 필터 구현

오늘 완성

상품 목록 · 검색 · 카테고리 선택 · 빈 결과 안내

현재 저장 범위

검색어 · 선택 카테고리는 새로고침하면 초기화
이후 저장 · 복원 기능을 연결해 유지 가능

2일차 1교시

장바구니 담기 · 수량 변경 · 총 수량 · 금액 계산

참고 자료와 실습 기준

수업 원자료

- 기존 교안 1~3회차를 1일차 4교시로 재구성
- 실제 검색 · 필터 프로젝트 실행 화면 사용

React 공식 문서

react.dev/learn/passing-props-to-a-component

react.dev/learn/rendering-lists

react.dev/reference/react/useState

react.dev/learn/you-might-not-need-an-effect

react.dev/learn/render-and-commit

react.dev/learn/react-developer-tools

JavaScript와 저장소: MDN

developer.mozilla.org

DOM · Semantic HTML

Array: map / filter / length

Destructuring · Logical AND / OR

Web APIs: Web Storage API

실습 기준

React Developer Tools 설치 후 Components 확인

Vite 개발 서버 주소 · 포트 사전 확인